The Agentic AI Builder's Checklist

1. Before you build — define the problem

DO

- Write down the exact task the agent performs, in one sentence, before touching any code.
- Name the person (or role) the agent serves and what "done right" looks like for them.
- List the inputs the agent gets and the outputs it must produce — with examples.
- Check that the problem is worth automating: it is repetitive, expensive, or slow when done by hand.
- Set a measurable success bar: "correct 95% of the time" beats "works well."
- Talk to the people who do the job today; they know the edge cases you don't.

DON'T

- Don't build an agent for a task you can't describe in plain words.
- Don't skip the "should this be software, not AI?" question — some problems are just forms and buttons.
- Don't start with a model API key; start with a written problem.

---

2. Designing the agent — scope, tools, autonomy

DO

- Start with the simplest thing that could work: one model call, one tool, a fixed sequence of steps.
- Give the agent more freedom only when the simple version demonstrably fails on real tasks.
- Design tools the way you'd design a good coworker's desk: a few clear, well-named tools beat dozens of vague ones.
- Write tool descriptions like instructions for a new hire — specific about what each tool does and returns.
- Decide up front which decisions the agent may make alone and which need a human's approval.
- Add stopping rules: maximum steps, time limits, and what happens when the agent gets stuck.

DON'T

- Don't hand the agent full autonomy on day one — earn it with evidence.
- Don't wrap every API endpoint as a separate tool; bundle related actions into one useful tool.
- Don't let the agent rewrite its own instructions or goals in production.
- Don't build a swarm of sub-agents before one agent works reliably.

---

3. Safety & trust — guardrails, human oversight, security

DO

- Put a human checkpoint before any irreversible action: sending, paying, deleting, diagnosing, publishing.
- Validate every tool input the agent produces, the same way you'd validate a web form.
- Treat user-provided content as untrusted data — never let it become instructions the agent follows blindly.
- Log every action the agent takes, so any outcome can be traced back and explained.
- Give the agent a clear, narrow identity and scope: "you help schedule visits" — not "you are a helpful assistant."
- Have a rollback plan for every action the agent can perform.

DON'T

- Don't let the agent act on text it read from the open web as if it were your instruction.
- Don't grant the agent broader permissions than the task needs — no admin keys for a read-only job.
- Don't hide the agent's reasoning from the user when a decision affects them; show your work.
- Don't ship without a way to turn the agent off quickly.

---

4. Testing & evaluation — scenarios, failure cases, metrics

DO

- Build a small set of realistic test scenarios (10–15) and run them after every change.
- Include failure cases on purpose: bad inputs, missing data, tool outages, contradictory instructions.
- Measure what matters: task success rate, cost per task, time per task — not just "it felt smart."
- Read full traces, not just final answers; most agent failures hide in the middle steps.
- Keep a test log of what you tried, what broke, and what fixed it.

DON'T

- Don't evaluate on two perfect demo inputs and call it tested.
- Don't assume a good demo run means a good agent — run it ten times and look at the variance.
- Don't let the same model that did the task be the only judge of whether it did it well.

---

5. Shipping & operating — observability, cost, iteration

DO

- Watch the agent's first real users closely; the first week teaches you more than the first month of building.
- Track token usage and latency per task from day one — agentic loops multiply both.
- Collect user corrections and turn the recurring ones into new test cases.
- Ship in stages: a pilot group, then wider — never everyone at once.
- Keep improving the boring parts: tool reliability, error messages, timeouts.

DON'T

- Don't launch to all users with the same settings you used in testing.
- Don't ignore cost until the invoice arrives — set budgets and alerts.
- Don't treat the agent as "done"; plan a regular review of its failures and near-misses.

General builder guidance — collected from public engineering practices at Anthropic, OpenAI, Google, LangChain, Hugging Face, and Microsoft. Not a judging rubric or scoring criteria.